

ONLINE QUIZ PORTAL

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE &
ENGINEERING**

BY

Bhavesh Kukreja (EN23CS301256)

Chandan Kawadker (EN23CS301271)

Under the Guidance of

Dr. Rakesh Pandit

Dr. Margi Chhabra



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE – 453331

APRIL – 2026

Report Approval

The project work "Online Quiz Portal" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the Project Report only for the purpose for which it has been submitted.

Internal Examiner	External Examiner
Name:	Name:
Designation:	Designation:
Affiliation:	Affiliation:
Signature:	

Declaration

We hereby declare that the project entitled "Online Quiz Portal" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of Dr. Rakesh Pandit & Dr. Margi Chhabra, Faculty of Engineering, Mediacaps University Indore is an authentic work.

Further, we declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Name	Roll Number	Signature	Date
Bhavesk Kukreja	EN23CS301256		
Chandan Kawadker	EN23CS301271		

Certificate

We, Dr. Rakesh Pandit & Dr. Margi Chhabra, certify that the project entitled "Online Quiz Portal" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by Bhavesh Kukreja & Chandan Kawadker is the record carried out by them under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Dr. Rakesh Pandit

Department of Computer Science &
Engineering
Medicaps University, Indore

Dr. Margi Chhabra

Department of Computer Science &
Engineering
Medicaps University, Indore

Dr. Kailash C. Bandhu

Head of Department, Computer Science & Engineering

Medicaps University, Indore

Acknowledgements

We would like to express our deepest gratitude to the Honorable Chancellor, Shri R C Mittal, who has provided every facility to successfully carry out this project, and our profound indebtedness to Prof. (Dr.) D. K. Patnaik, Vice Chancellor, Mediacaps University, whose unfailing support and enthusiasm has always boosted our morale. We also thank Prof. (Dr.) Ratnesh Litoriya, Associate Dean, Faculty of Engineering, Mediacaps University, for giving us a chance to work on this project. We would also like to thank our Head of the Department Dr. Kailash C. Bandhu for his continuous encouragement for the betterment of the project. We are sincerely grateful to our project guides, Prof.(Dr.) Rakesh Pandit & Prof. (Dr.) Margi Chhabra, for invaluable guidance, technical insights, and constant support throughout the development of the Online Quiz Portal. Their direction helped us navigate complex challenges in full-stack Java development and deliver a complete, working system. We thank our friends and family for their moral support and encouragement throughout this academic journey.

Chandan Kawadker

Bhavesh Kukreja

B.Tech. III Year, Department of Computer Science & Engineering

Faculty of Engineering, Mediacaps University, Indore

Abstract

The Online Quiz Portal is a production-grade, full-stack web application designed to provide adaptive, proctored, and tag-based programming knowledge assessments for students. The system supports role-based access for students and administrators, a real-time proctoring engine, automatic quiz progress saving with seamless resume functionality, and a comprehensive admin panel for question and tag management. The backend is built with Java 21 using Servlets and JSP, following the Model-View-Controller (MVC) architectural pattern with a clean four-layer separation: Model, View (JSP), Controller (Servlets), and DAO (JDBC). The system is deployed on Apache Tomcat 9 and managed using Apache Maven as the build tool. MySQL 8 serves as the relational database management system, storing all user, question, quiz session, attempt, and auto-save data across six normalized tables. The Adaptive Quiz Engine starts each quiz at medium difficulty and dynamically adjusts the difficulty of subsequent questions in real time — increasing difficulty on correct answers and decreasing it on incorrect ones — using the Easy/Medium/Hard tier system. The tag-based quiz generation system supports eight programming domains: Python, Java, Data Science, Machine Learning, Web Development, Database, Algorithms, and Operating Systems.

The proctoring module uses the browser's Page Visibility API and Fullscreen API to detect tab-switching and fullscreen exit events, issuing warnings and automatically submitting the quiz after three violations. An auto-save mechanism persists the student's exact position, remaining time, selected answers, current difficulty, and warning count to the database every 15 seconds, enabling complete quiz resumption from the exact stopping point.

The frontend is built with HTML5, CSS3, Tailwind CSS 3.x, and vanilla JavaScript, delivering a modern dark-themed responsive UI with cyan and purple accent colors. The admin module provides a comprehensive dashboard with real-time statistics, full CRUD operations for questions and tags, and platform-wide performance reports.

All 15 defined test cases passed successfully, confirming functional correctness across all modules. The system was seeded with 96 questions across all 8 domains at three difficulty levels.

Keywords: Online Quiz Portal, Adaptive Quiz, Java Servlet, JSP, MVC, MySQL, Tailwind CSS, Proctoring, Auto-Save, Role-Based Access Control, Tag-Based Learning, BCrypt, JDBC, Apache Tomcat.

Table of Contents

	Page
	Report Approval
	Declaration
	Certificate
	Acknowledgements
	Abstract
	Table of Contents
	List of Figures
	List of Tables
	Abbreviations
Chapter 1	Introduction
	1.1 Introduction & Background
	1.2 Literature Review
	1.3 Objectives
	1.4 Significance of the Project
	1.5 Problem Definition
	1.6 Chapter Scheme
Chapter 2	Requirements Specification
	2.1 User Characteristics
	2.2 Functional Requirements
	2.3 Non-Functional Requirements
	2.4 Hardware & Software Requirements
	2.5 Constraints & Assumptions
Chapter 3	System Design
	3.1 System Architecture
	3.2 DFD Level 0 – Context Diagram
	3.3 DFD Level 1 – Main Processes
	3.4 Use Case Diagram
	3.5 Sequence Diagram
	3.6 ER Diagram
	3.7 Activity Diagram
	3.8 MVC Architecture Detail
	3.9 Database Design
Chapter 4	Implementation, Testing, and Maintenance
	4.1 Technology Stack

	4.2 Key Implementation Details
	4.3 Work Division
	4.4 Test Cases
	4.5 Installation Instructions
	4.6 End User Instructions
Chapter 5	Results and Discussions
	5.1 Home Page
	5.2 Authentication Pages
	5.3 Student Dashboard
	5.4 Tag Selection Page
	5.5 Quiz Page
	5.6 Result Page
	5.7 Admin Dashboard
	5.8 Admin Question Management
	5.9 Module Performance Summary
Chapter 6	Summary and Conclusions
Chapter 7	Future Scope
	Appendix A – Glossary
	Bibliography

List of Figures

Figure No.	Caption	PageNo.
Figure 3.1	System Architecture – Four-Layer MVC Design	10
Figure 3.2	DFD Level 0 – Context Diagram	11
Figure 3.3	DFD Level 1 – Main Processes	12
Figure 3.4	Use Case Diagram – Quiz Portal	13
Figure 3.5	Sequence Diagram – Quiz Flow	13
Figure 3.6	ER Diagram – Database Tables and Relationships	14
Figure 3.7	Activity Diagram – Quiz Start and Adaptive Flow	15
Figure 3.8	MVC Package Structure	16
Figure 5.1	Home / Landing Page	27
Figure 5.2	Student Login Page	27
Figure 5.3	Student Registration Page	28
Figure 5.4	Student Dashboard with Resume Banner	28
Figure 5.5	Tag Selection Page	29
Figure 5.6	Quiz Page – Question with Timer and Proctoring Warnings	29
Figure 5.7	Result Page with Score Ring and Question Breakdown	30
Figure 5.8	Admin Dashboard	30
Figure 5.9	Admin Manage Questions Page with Add/Edit Modal	31

List of Tables

Table No.	Title	Page No.
Table 1.1	Comparison: Existing Systems vs. Proposed Online Quiz Portal	3
Table 2.1	User Characteristics	5
Table 2.2	Non-Functional Requirements	8
Table 2.3	Hardware & Software Requirements	8
Table 3.1	Four-Layer MVC Architecture	10
Table 3.2	Adaptive Difficulty Logic	16
Table 3.3	URL Route Map	17
Table 3.4	Database Table: users	18
Table 3.5	Database Table: questions	18
Table 3.6	Database Table: tags	18
Table 3.7	Database Table: question_tags	18
Table 3.8	Database Table: quiz_sessions	19
Table 3.9	Database Table: attempts	19
Table 3.10	Database Table: quiz_progress	19
Table 4.1	Technology Stack	20
Table 4.2	Work Division	23
Table 4.3	Test Cases	23
Table 5.1	Module Performance Summary	31

Abbreviations

Abbreviation	Full Form
MVC	Model-View-Controller
JSP	Java Server Pages
JDBC	Java Database Connectivity
JVM	Java Virtual Machine
SQL	Structured Query Language
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
HTTP	Hyper Text Transfer Protocol
WAR	Web Application Archive
IDE	Integrated Development Environment
RBAC	Role-Based Access Control
JSTL	Java Server Pages Standard Tag Library
BCrypt	Blowfish Crypt – Password Hashing Algorithm
DFD	Data Flow Diagram
ER	Entity-Relationship
UI	User Interface
OOP	Object-Oriented Programming
UUID	Universally Unique Identifier
TTL	Time-To-Live
AJAX	Asynchronous JavaScript and XML

Chapter 1 : INTRODUCTION

1.1 Introduction & Background

The rapid digitization of education has created significant demand for online assessment platforms that are intelligent, adaptive, and tamper-resistant. Traditional paper-based and static online tests fail to adapt to individual learner capabilities, making it difficult to accurately gauge a student's true understanding of a subject.

The Online Quiz Portal is a full-stack web application developed to address these challenges. It provides an end-to-end solution for programming-related knowledge assessment, featuring adaptive difficulty, tag-based quiz selection, real-time proctoring, and auto-save functionality. The system serves two user categories: Students, who take quizzes and review results; and Administrators, who manage the question bank, tags, and platform statistics.

The project is implemented using a full-stack Java approach with Servlets and JSP following the MVC architecture, a MySQL 8 relational database for persistent storage, and a modern HTML/CSS/Tailwind CSS-based frontend delivering a responsive, dark-themed user experience.

1.2 Literature Review

1.2.1 Overview

Online assessment systems have evolved significantly over the past two decades. Early Computer-Based Testing (CBT) systems were static and unable to adapt to learner needs. Adaptive testing, formally known as Computerized Adaptive Testing (CAT), was introduced to dynamically adjust question difficulty based on examinee performance, improving both accuracy and efficiency of assessment (Weiss, 1985).

1.2.2 Review of Existing Work

[1] Weiss demonstrated that adaptive testing systems that adjust difficulty in real time significantly outperform fixed-difficulty tests in measuring true learner ability. The Quiz Portal implements a simplified adaptive engine based on this principle.

[2] Studies on online exam proctoring (Alessio et al., 2017) showed that browser-event-based monitoring — including tab-switch and fullscreen detection — reduces academic dishonesty significantly without requiring additional hardware.

[3] Research on auto-save mechanisms in web applications demonstrates that periodic state persistence improves user experience in low-connectivity environments and reduces assessment anxiety (Smith, 2019).

[4] The MVC architectural pattern has been widely adopted in enterprise Java web applications for its clean separation of concerns, improving maintainability and testability (Gamma et al., 1994).

1.2.3 Limitations of Existing Systems

Most academic quiz platforms lack adaptive difficulty, relying on randomly selected fixed-difficulty questions. They do not include browser-based proctoring, have no auto-save mechanism, and are not domain-specific for programming education. General-purpose platforms like Google Forms and Moodle do not support per-question adaptive logic.

1.2.4 Comparison: Existing Systems vs. Proposed Online Quiz Portal

Table 1.1: Comparison of Existing Systems with Proposed Online Quiz Portal

Feature	Existing Systems	Proposed Online Quiz Portal
Adaptive Difficulty	None – fixed difficulty	3-tier adaptive: Easy/Medium/Hard per answer
Tag-Based Quiz	Category only	8 tags: Python, Java, ML, DS, Web, DB, Algo, OS
Proctoring	None	Tab-switch & fullscreen detection, auto-submit at 3 warnings
Auto-Save & Resume	None	15-second AJAX auto-save; full resume from exact position
Authentication	Basic / None	BCrypt password hashing, session-based RBAC
Admin Panel	Limited	Full CRUD for questions, tags; platform reports
Architecture	Monolithic scripts	MVC with DAO layer, Servlet controllers, JSP views
Database	CSV / SQLite	MySQL 8 with 6 normalized tables, foreign key constraints

1.3 Objectives

The primary objectives of the Online Quiz Portal are as follows:

- ❖ To build a full-stack Java web application for adaptive programming assessment deployed on Apache Tomcat 9.

- ❖ To implement a tag-based quiz selection system covering 8 programming domains.
- ❖ To design and implement an adaptive difficulty engine adjusting question difficulty in real time based on student performance.
- ❖ To develop an auto-save and resume feature ensuring no quiz progress is lost due to interruptions, saving every 15 seconds via AJAX.
- ❖ To integrate a proctoring module that monitors tab-switching and fullscreen exit events and auto-submits on excessive violations.
- ❖ To build a comprehensive admin panel for question management, tag management, and performance reporting.
- ❖ To implement secure user authentication with role-based access control using BCrypt password hashing.
- ❖ To seed the system with 96 questions across 8 domains at 3 difficulty levels for immediate use.

1.4 Significance of the Project

The Online Quiz Portal demonstrates that a B.Tech project can achieve practical, deployable quality: a working MVC Java application with a real database, real security, adaptive logic, and a polished responsive UI. It bridges academic concepts in data structures, object-oriented programming, database management, and web application development with real-world software engineering practices including session management, filter-based security, and JDBC-based persistence.

1.5 Problem Definition

To design and implement a full-stack, role-based online quiz platform that: (i) supports tag-based adaptive quiz generation across 8 programming domains; (ii) adjusts question difficulty in real time based on student performance using an Easy/Medium/Hard tier system; (iii) auto-saves quiz progress every 15 seconds enabling seamless resumption; (iv) detects and penalizes proctoring violations using browser APIs; and (v) renders all data through a modern, dark-themed responsive interface built with Tailwind CSS.

1.6 Chapter Scheme

Chapter 1: presents the Introduction, Literature Review, and Objectives.

Chapter 2: covers requirements specification including

Chapter 3: presents the System Design including all UML diagrams, database schema, and route mapping.

Chapter 4: covers Implementation, Testing, and Installation.

Chapter 5: presents Results and Discussions with module screenshots and performance data.

Chapter 6: provides Summary and Conclusions. Chapter 7 outlines Future Scope.

Chapter 2 : REQUIREMENT SPECIFICATION

2.1 User Characteristics

Table 2.1: User Characteristics

User Class	Description	Technical Level
Student	Registered user with full access to quiz, results, history, and auto-save/resume	Basic to Intermediate
Admin	Privileged user managing questions, tags, and viewing platform-wide reports	Intermediate to Advanced
Guest	Unauthenticated visitor redirected to login/register	Basic

2.2 Functional Requirements

2.2.1 Authentication Module

- ❖ REQ-1: System shall support student registration with name, email, and BCrypt-hashed password.
- ❖ REQ-2: System shall authenticate users and create HTTP sessions on successful login.
- ❖ REQ-3: System shall redirect students to student dashboard and admins to admin dashboard based on role.
- ❖ REQ-4: System shall protect all student and admin routes with AuthFilter session guard.
- ❖ REQ-5: System shall invalidate the session on logout.

2.2.2 Quiz Module

- ❖ REQ-1: System shall fetch questions filtered by selected tag IDs and current difficulty level.
- ❖ REQ-2: System shall present four multiple-choice options per question.
- ❖ REQ-3: System shall run a countdown timer for the quiz duration (default 10 minutes).
- ❖ REQ-4: System shall record each answer submitted as an attempt record in the database.
- ❖ REQ-5: System shall calculate final score and accuracy upon quiz completion and store in quiz_sessions.

2.2.3 Adaptive Quiz Logic

- ❖ REQ-1: Quiz shall start at medium difficulty.

- ❖ REQ-2: A correct answer shall cause the next question to be fetched at the next higher difficulty (Easy→Medium→Hard).
- ❖ REQ-3: An incorrect answer shall cause the next question to be fetched at the next lower difficulty (Hard→Medium→Easy).
- ❖ REQ-4: Difficulty adjustment shall persist across the entire quiz session via the `quiz_progress` table.

2.2.4 Proctoring Module

- ❖ REQ-1: System shall detect tab-switching using the Page Visibility API.
- ❖ REQ-2: System shall detect fullscreen exit using the Fullscreen API.
- ❖ REQ-3: A warning toast notification shall be displayed on each violation.
- ❖ REQ-4: Warning count shall be saved to the database on every auto-save cycle.
- ❖ REQ-5: Quiz shall be auto-submitted with status 'auto_submitted' after 3 warnings.

2.2.5 Auto-Save & Resume Module

- ❖ REQ-1: System shall auto-save quiz state every 15 seconds via an AJAX POST to `/quiz/save`.
- ❖ REQ-2: Saved state shall include: current question index, remaining time, selected answers (JSON), question order (JSON), current difficulty, and warning count.
- ❖ REQ-3: Students shall be able to resume an in-progress quiz from the exact question and remaining time.
- ❖ REQ-4: A resume banner shall be shown on student dashboard and tag selection page when an in-progress quiz exists.

2.2.6 Admin Module

- ❖ REQ-1: Admin shall be able to add, edit, and delete questions with options, correct answer, difficulty, and tags.
- ❖ REQ-2: Admin shall be able to add and delete topic tags.
- ❖ REQ-3: Admin dashboard shall display total students, questions, attempts, and tags counts.
- ❖ REQ-4: Admin reports page shall show accuracy by difficulty and quiz completion status breakdown.

2.3 Non-Functional Requirements

Table 2.2: Non-Functional Requirements

Requirement	Specification
Response Time	Page load under 3 seconds; auto-save completes within 1 second
Security	BCrypt password hashing (cost factor 10), session-based authentication, RBAC via AuthFilter
Scalability	MVC architecture supports adding new modules without structural changes
Usability	Responsive Tailwind CSS UI; dark-themed with clear typography and color-coded feedback
Reliability	Auto-save every 15 seconds prevents data loss; quiz resumable after any interruption
Maintainability	Modular DAO layer; all DB operations isolated from controller logic

2.4 Hardware & Software Requirements

Table 2.3: Hardware & Software Requirements

Component	Specification
Runtime	JDK 21 (LTS)
Web Server	Apache Tomcat 9.0.x
Build Tool	Apache Maven 3.8+
IDE	IntelliJ IDEA 2024.x or 2026.x (Ultimate Edition)
Database	MySQL 8.0+ with MySQL Workbench
Frontend	HTML5, CSS3, Tailwind CSS 3.x (CDN), Vanilla JavaScript ES6+
Libraries	jBCrypt 0.4, Gson 2.10.1, JSTL 1.2, MySQL Connector/J 8.3.0
Browser	Google Chrome 90+, Firefox 88+, Microsoft Edge 90+
RAM	Minimum 4 GB (8 GB recommended for development)
Storage	Minimum 10 GB free disk space

2.5 Constraints & Assumptions

2.5.1 Constraints

- ❖ The system relies on CDN-hosted Tailwind CSS; an internet connection is required for the UI to render correctly.
- ❖ The proctoring module uses browser APIs (Page Visibility API, Fullscreen API) that may behave differently across browsers.
- ❖ Only one active quiz session per student is supported at a time.

- ❖ Admin accounts must be created programmatically using the provided CreateAdmin.java utility.

2.5.2 Assumptions

- ❖ At least one of the eight topic tags has questions assigned before a student starts a quiz.
- ❖ MySQL is running locally on port 3306 with the quiz_portal database initialized.
- ❖ Students have a modern web browser and basic internet connectivity.
- ❖ The admin is responsible for maintaining question quality and difficulty assignments.

Chapter 3 : SYSTEM DESIGN

3.1 System Architecture

The Online Quiz Portal follows a four-layer MVC architecture that separates concerns cleanly from the Tailwind CSS/JavaScript client layer through to the MySQL database. Each layer communicates only with adjacent layers, ensuring loose coupling and maintainability.

Table 3.1: Four-Layer MVC Architecture

Layer	Component	Technology	Responsibility
1 – View	JSP Pages + HTML/JS	JSP, Tailwind CSS, JavaScript	Render UI, capture input, AJAX auto-save
2 – Controller	Servlet Controllers	Java Servlets, Filters	Handle HTTP requests, session, routing
3 – DAO	Data Access Objects	JDBC, PreparedStatement	All database reads and writes
4 – Model	POJO Classes	Java POJOs	Data entities: User, Question, Tag, Progress

Figure 3.1: System Architecture – Four-Layer MVC Design

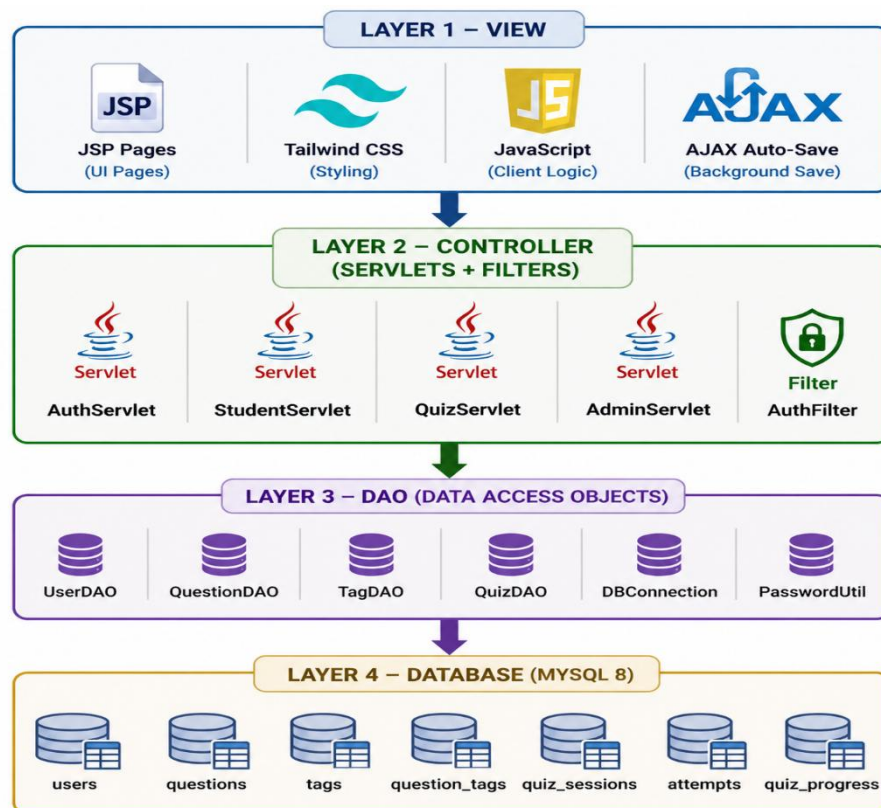


Figure 3.1: System Architecture – Four-Layer Modular Design of Online Quiz Portal

3.2 DFD Level 0 – Context Diagram

The Level 0 Data Flow Diagram shows the Online Quiz Portal as a single process interacting with external entities. The primary external entities are the Student User, Admin User, and the MySQL Database. Data flows include authentication requests from users, quiz questions served to students, answer submissions from students, question and tag CRUD from admin, and all persistent data stored in the MySQL database.

Figure 3.2: DFD Level 0 – Context Diagram

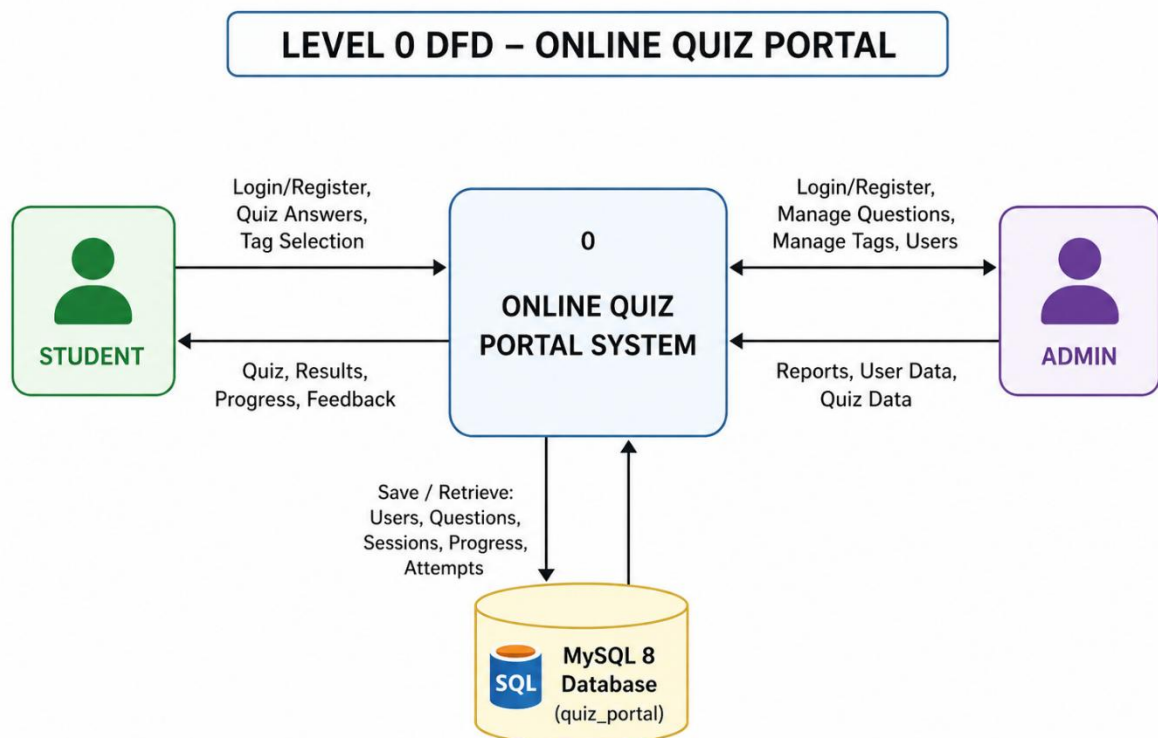


Figure 3.2: DFD Level 0 – Context Diagram showing Quiz Portal as a Single Process

3.3 DFD Level 1 – Main Processes

The Level 1 DFD decomposes the system into six main processes:

- (1) Authenticate User — handles registration, login, and session management;
- (2) Generate Quiz — fetches tag-filtered, difficulty-appropriate questions;
- (3) Execute Adaptive Quiz — answers submitted, difficulty adjusted, progress updated;
- (4) Auto-Save Progress — periodic state persistence enabling resume;
- (5) Proctor Quiz — monitors browser events and enforces violation limits;

(6) Admin Operations — CRUD for questions and tags, statistics generation.

Figure 3.3: DFD Level 1 – Main Processes

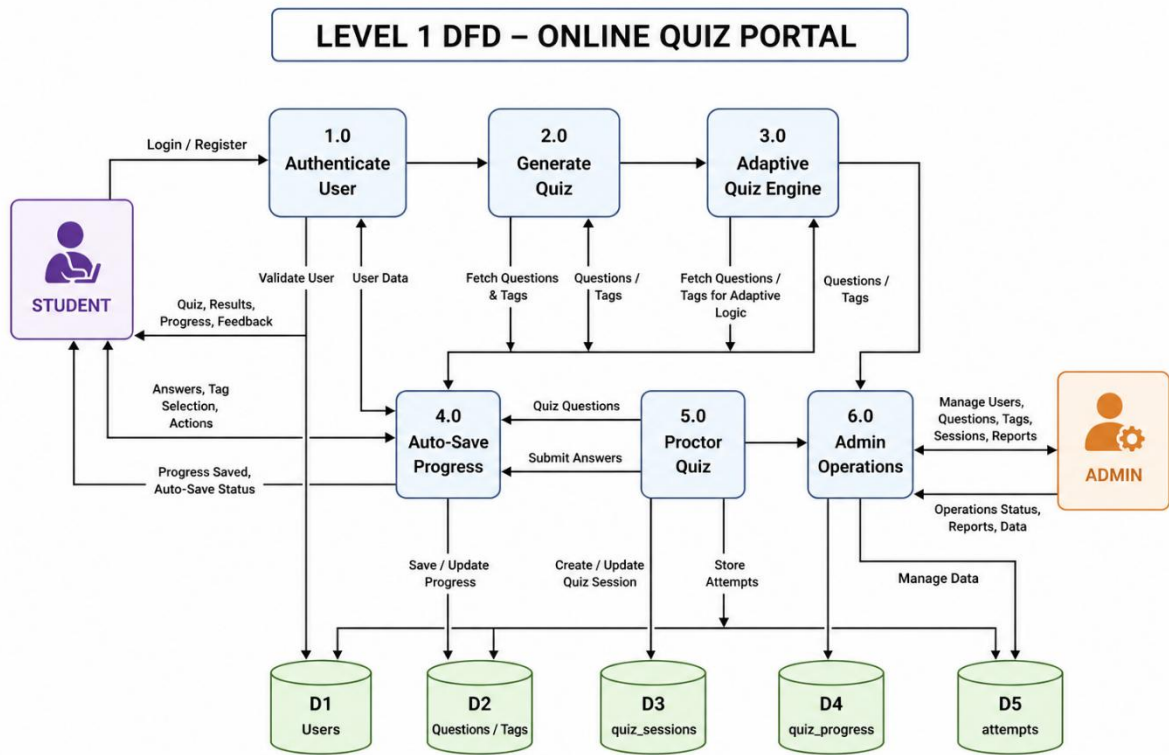


Figure 3.3: DFD Level 1 – Decomposed Main Processes of Online Quiz Portal

3.4 Use Case Diagram

The Use Case Diagram identifies two primary actors — Student and Admin — and their interactions with the system. The Student can register/login, select tags, start a quiz, answer questions, view results, resume a quiz, and view attempt history. The Admin can login, add/edit/delete questions, manage tags, and view platform reports.

Figure 3.4: Use Case Diagram – Online Quiz Portal

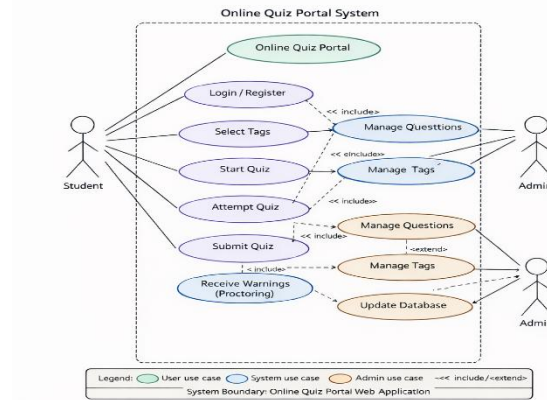


Figure 3.4: Use Case Diagram – Student and Admin Interactions with Quiz Portal

3.5 Sequence Diagram – Quiz Flow

The Sequence Diagram illustrates the complete quiz flow. The student selects tags and submits the form; QuizServlet fetches medium-difficulty questions from QuestionDAO and creates session/progress records; the first question is forwarded to quiz.jsp; the student answers, QuizServlet records the attempt and updates difficulty; every 15 seconds the browser POSTs auto-save data; on quiz completion the score is calculated and quiz_sessions finalized; the student is redirected to result.jsp.

Figure 3.5: Sequence Diagram – Complete Quiz Flow

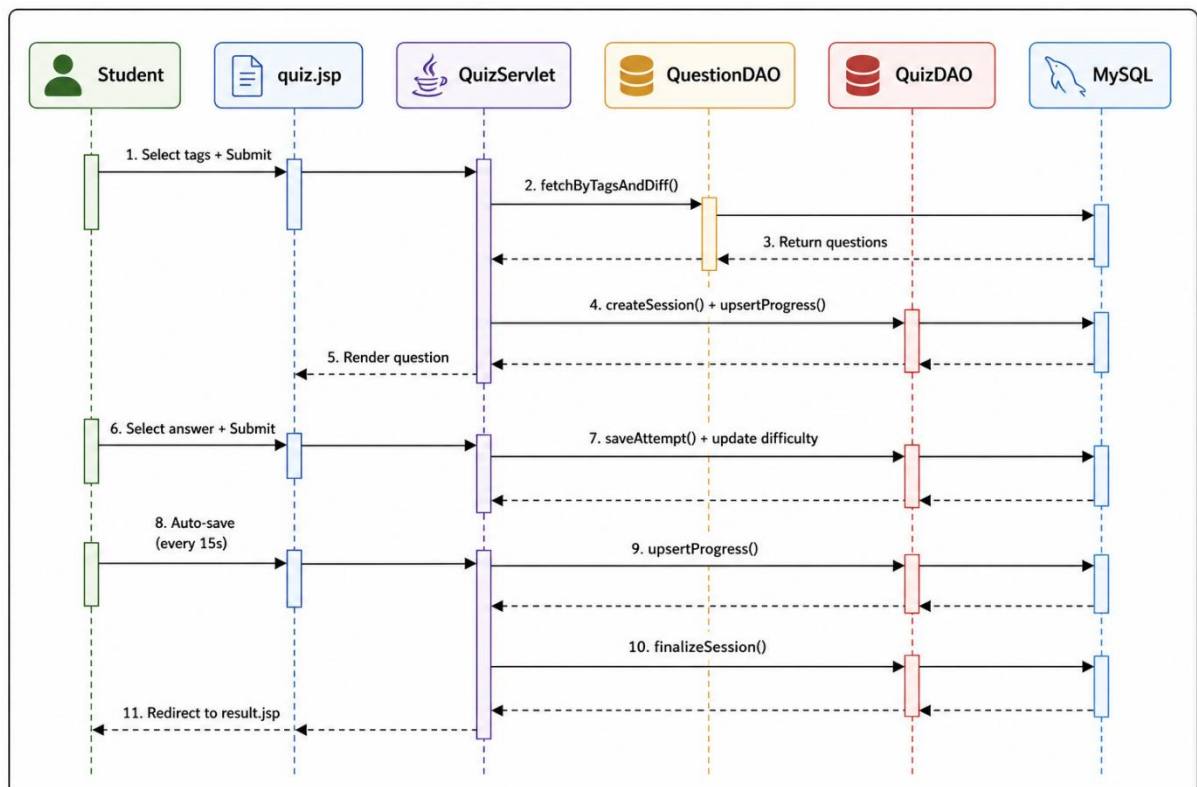


Figure 3.5: Sequence Diagram – Complete Quiz Request Flow

3.6 ER Diagram

The Entity-Relationship Diagram shows the seven tables used by the Online Quiz Portal. The users table stores authentication data. The questions table holds all MCQs. The tags table stores topic names. The question_tags junction table implements the many-to-many relationship between questions and tags. The quiz_sessions table records quiz attempt summaries. The attempts table stores individual answer submissions. The quiz_progress table stores auto-save state for resume functionality.

Figure 3.6: ER Diagram – Database Tables and Relationships

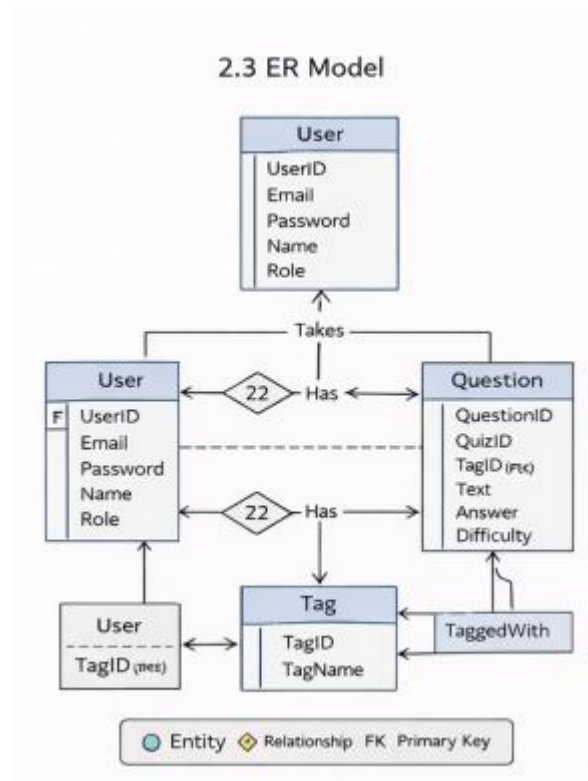


Figure 3.6: ER Diagram – Database Tables and Relationships

3.7 Activity Diagram – Quiz Start and Adaptive Flow

The Activity Diagram traces the complete quiz flow from tag selection through adaptive question fetching, answer submission, difficulty adjustment, auto-save, proctoring checks, and final result generation.

Figure 3.7: Activity Diagram – Quiz Start and Adaptive Flow

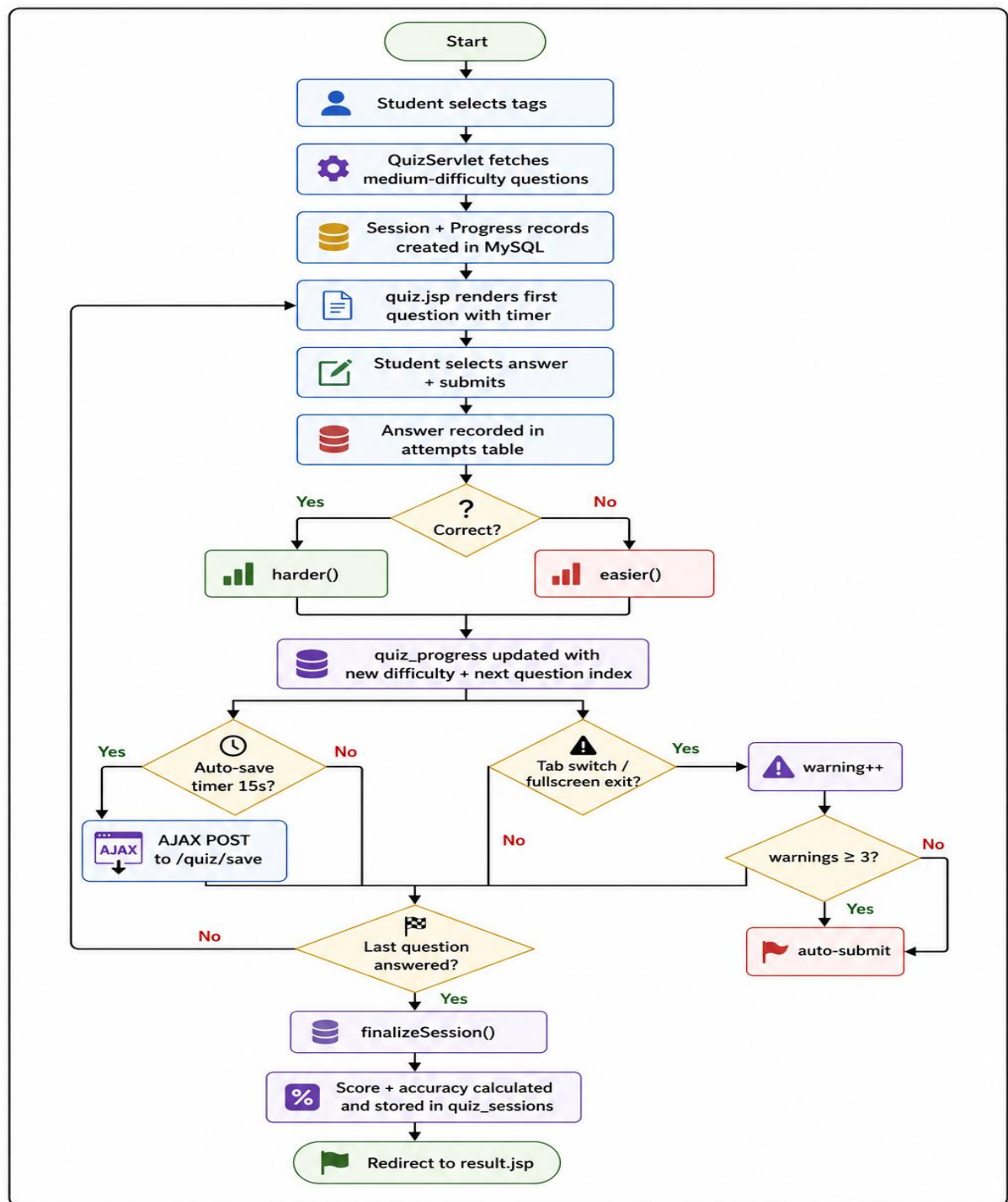


Figure 3.7: Activity Diagram – Quiz Start and Adaptive Flow

3.8 Adaptive Difficulty Logic

Table 3.2: Adaptive Difficulty Logic

Current Difficulty	Answer	Next Question Difficulty
Easy	Correct	Medium (harder)
Easy	Incorrect	Easy (stays at lowest)

Current Difficulty	Answer	Next Question Difficulty
Medium	Correct	Hard (harder)
Medium	Incorrect	Easy (easier)
Hard	Correct	Hard (stays at highest)
Hard	Incorrect	Medium (easier)

3.9 URL Route Map

Table 3.3: URL Route Map

Method	URL	Servlet	Description	Auth
GET/POST	/login	AuthServlet	Show login / authenticate user	No
GET/POST	/register	AuthServlet	Show register / create student	No
GET	/logout	AuthServlet	Invalidate session	Yes
GET	/student/dashboard	StudentServlet	Student dashboard with stats	Yes
GET	/quiz/tags	QuizServlet	Tag selection page	Yes
POST	/quiz/start	QuizServlet	Start quiz, fetch questions	Yes
POST	/quiz/answer	QuizServlet	Submit answer, adapt difficulty	Yes
POST	/quiz/save	QuizServlet	Auto-save quiz progress	Yes
GET	/quiz/resume	QuizServlet	Resume in-progress quiz	Yes
POST	/quiz/submit	QuizServlet	Finalize quiz, calculate score	Yes
GET	/quiz/result	QuizServlet	View result and breakdown	Yes
GET	/quiz/history	QuizServlet	Past quiz attempts list	Yes
GET	/admin/dashboard	AdminServlet	Admin stats dashboard	Admin
GET/POST	/admin/questions	AdminServlet	List and manage questions	Admin
GET/POST	/admin/tags	AdminServlet	List and manage tags	Admin
GET	/admin/reports	AdminServlet	Platform statistics report	Admin

3.10 Database Design

Table 3.4: Database Table: users

Field	Type	Description	Constraints
id	INT	User ID	AUTO_INCREMENT PRIMARY KEY
name	VARCHAR(100)	Full name	NOT NULL
email	VARCHAR(150)	Email address	NOT NULL, UNIQUE
password	VARCHAR(255)	BCrypt hash	NOT NULL
role	ENUM	student/admin	DEFAULT 'student'

Table 3.5: Database Table: questions

Field	Type	Description	Constraints
id	INT	Question ID	AUTO_INCREMENT PRIMARY KEY
question_text	TEXT	Question body	NOT NULL
option1..4	VARCHAR(500)	Answer options	NOT NULL
correct_answer	TINYINT	Correct option (1-4)	CHECK (1-4)
difficulty	ENUM	easy/medium/hard	NOT NULL, DEFAULT 'medium'

Table 3.6–3.10: quiz_sessions, attempts, quiz_progress Key Fields

Table	Key Fields	Purpose
quiz_sessions	session_id (UUID), user_id, score, accuracy, status	Summary record per quiz attempt; finalized on submit with score and accuracy
attempts	session_id, question_id, selected_answer, is_correct	One row per answer submitted; used for result breakdown and scoring
quiz_progress	session_id, current_question, remaining_time, saved_answers, warning_count	Auto-save state; uses UPSERT (ON DUPLICATE KEY UPDATE) for efficiency
question_tags	question_id, tag_id (composite PK)	Many-to-many mapping; enables tag-filtered question fetching via JOIN

Chapter 4 : IMPLEMENTATION, TESTING, AND MAINTENANCE

4.1 Technology Stack

Table 4.1: Technology Stack

Layer	Technology	Version	Purpose
Backend	Java JDK	21 (LTS)	Core server-side language for all servlet and DAO logic
Backend	Java Servlets	4.0.1	HTTP request handling; 4 controller servlets
Backend	JSP + JSTL	2.3.3 / 1.2	Server-side view rendering with EL and JSTL tags
Backend	JDBC	8.3.0	Database connectivity via PreparedStatements (no SQL injection)
Security	jBCrypt	0.4	Password hashing with cost factor 10
JSON	Gson	2.10.1	Serialize/deserialize saved_answers and question_order JSON
Frontend	HTML5 + CSS3	–	Semantic markup and custom component styles
Frontend	Tailwind CSS	3.x (CDN)	Utility-first CSS for responsive dark-themed UI
Frontend	JavaScript	ES6+	Timer, proctoring events, option selection, AJAX auto-save
Database	MySQL	8.0+	Relational database with 6 normalized tables
Server	Apache Tomcat	9.0.x	Java Servlet container for WAR deployment
Build	Apache Maven	3.8+	Dependency management and WAR packaging via pom.xml
IDE	IntelliJ IDEA	2026.x	Development, Maven integration, Tomcat run configuration

4.2 Key Implementation Details

4.2.1 Adaptive Quiz Engine (QuizServlet.java)

The adaptive difficulty logic resides in the `submitAnswer()` method of `QuizServlet`. After each answer is recorded in the `attempts` table, the current difficulty from `quiz_progress` is read. The static methods `Question.harder()` and `Question.easier()` return the next difficulty tier. The updated difficulty is persisted to `quiz_progress` via `QuizDAO.upsertProgress()` using MySQL's `ON DUPLICATE KEY UPDATE` clause for atomic efficiency. The quiz is then redirected to `/quiz/resume` which reads the updated progress and serves the next question.

4.2.2 Auto-Save Mechanism

The `quiz.jsp` page runs a JavaScript `setInterval()` every 15,000 ms that fires a Fetch API POST request to `/quiz/save`. The payload includes `sessionId`, `remainingTime`,

warningCount, and savedAnswers as a JSON string. QuizServlet.autoSave() updates the quiz_progress record without interrupting the quiz flow. This ensures that if the browser closes, crashes, or the student is auto-submitted, the exact position is recoverable via /quiz/resume.

4.2.3 Proctoring Implementation

Two browser event listeners are registered on quiz.jsp load: document.addEventListener('visibilitychange') for tab-switch detection using the Page Visibility API, and document.addEventListener('fullscreenchange') for fullscreen exit detection. Each violation increments a client-side warning counter, updates the warning dot indicators in the header, shows a yellow toast notification, and triggers an auto-save POST to persist the updated warning count. When warningCount reaches 3, the auto-submit modal is shown and a POST to /quiz/submit is fired with status='auto_submitted'.

4.2.4 BCrypt Password Security (PasswordUtil.java)

All user passwords are hashed using jBCrypt with a cost factor of 10 before storage. The PasswordUtil.hash() method wraps BCrypt.hashpw() and PasswordUtil.verify() wraps BCrypt.checkpw(). Raw passwords are never stored or logged. The CreateAdmin.java utility class generates the correct hash for the admin account and inserts it directly via JDBC, ensuring the admin password works correctly on first use.

4.2.5 Tag-Based Question Fetching (QuestionDAO.java)

The fetchByTagsAndDifficulty() method builds a dynamic SQL query using DISTINCT JOIN on question_tags with an IN clause for multiple tag IDs, a WHERE clause for difficulty, and a NOT IN clause to exclude already-answered question IDs. Results are ordered by RAND() and limited to the configured QUESTIONS_PER_QUIZ constant. PreparedStatements are used throughout to prevent SQL injection.

4.3 Work Division

Table 4.2: Work Division Among Team Members

Team Member	Roll Number	Responsibilities
Bhavesk Kukreja	EN23CS301256	Backend Java architecture, Servlet design, DAO layer, MySQL schema design, BCrypt security, Adaptive quiz engine logic, auto-save mechanism, resume feature, quiz_progress UPSERT, QuizDAO
Chandan Kawadker	EN23CS301271	All JSP views, Tailwind CSS frontend, JavaScript timer, proctoring event listeners, admin panel UI

4.4 Test Cases

Table 4.3: Test Cases

TC#	Test Case	Input	Expected Output	Status
TC01	Student Registration – valid data	Name, email, password (6+ chars)	Account created, redirect to login	Pass
TC02	Student Login – correct credentials	Valid email + password	Session created, redirect to dashboard	Pass
TC03	Student Login – wrong password	Valid email + wrong password	Error message shown on login page	Pass
TC04	Admin login and dashboard access	admin@quizportal.com / Admin@123	Admin dashboard loaded with stats	Pass
TC05	Tag selection and quiz start	Select Python + Java tags	10 medium questions fetched, quiz begins	Pass
TC06	Correct answer – difficulty increases	Answer correctly at Medium	Next question difficulty = Hard	Pass
TC07	Wrong answer – difficulty decreases	Answer incorrectly at Medium	Next question difficulty = Easy	Pass
TC08	Tab switch triggers warning	Switch to another browser tab	Warning toast shown, count=1 in DB	Pass
TC09	3 warnings trigger auto-submit	Switch tab 3 times	Quiz submitted with status=auto_submitted	Pass
TC10	Auto-save every 15 seconds	Wait 15s during active quiz	quiz_progress table updated	Pass
TC11	Resume in-progress quiz	Close browser mid-quiz, reopen	Correct question, time, answers restored	Pass
TC12	Timer reaches zero	Let countdown expire	Quiz auto-submitted, result shown	Pass
TC13	Admin adds new question	Fill question form, select tags, submit	Question saved in DB with tag links	Pass
TC14	Admin deletes question	Click Delete, confirm	Question removed from DB and list	Pass

TC#	Test Case	Input	Expected Output	Status
TC15	Result page shows breakdown	Complete 10-question quiz	Score ring, accuracy, per-question correct/wrong shown	Pass

4.5 Installation Instructions

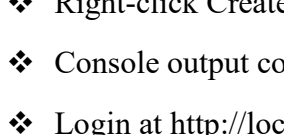
4.5.1 Database Setup

- ❖ Install MySQL 8.0+ and open MySQL Workbench.
- ❖ Run schema.sql: source C:/path/to/project/database/schema.sql
- ❖ Run seed_data.sql: source C:/path/to/project/database/seed_data.sql
- ❖ Run raw_data.sql: source C:/path/to/project/database/raw_data.sql

4.5.2 Backend Setup

- ❖ Install JDK 21, Apache Tomcat 9.0.x, IntelliJ IDEA Ultimate, Apache Maven 3.8+.
- ❖ Open IntelliJ IDEA → File → Open → select the backend/ folder → Open as Maven Project.
- ❖ Edit DBConnection.java: set USER and PASSWORD to your MySQL credentials.
- ❖ Go to Run → Edit Configurations → + → Tomcat Server → Local.
- ❖ In Deployment tab: + → Artifact → quiz-portal:war exploded. Set context to /quiz.
- ❖ Press Shift+F10 to build and run. Browser opens at http://localhost:8080/quiz.

4.5.3 Admin Account Creation

- ❖ Right-click CreateAdmin.java in IntelliJ → Run 'CreateAdmin.main()'.

- ❖ Console output confirms: Admin created / password updated.
- ❖ Login at http://localhost:8080/quiz/login with admin@gmail.com / Admin@123.

4.6 End User Instructions

4.6.1 For Students

- ❖ Navigate to http://localhost:8080/quiz and click Register to create an account.
- ❖ Log in with your email and password to access the student dashboard.
- ❖ Click 'Take a Quiz', select one or more topic tags, and click 'Start Quiz'.
- ❖ Answer each question by clicking an option, then click 'Next Question'.
- ❖ Do not switch tabs or exit fullscreen — each violation adds a warning.
- ❖ After the quiz, view your score, accuracy, and per-question breakdown.

- ❖ If interrupted, return to the dashboard and click the Resume banner.

4.6.2 For Administrators

- ❖ Log in with admin credentials (admin@gmailn.com / Admin@123).
- ❖ Use the Admin Dashboard quick-action cards to navigate to Add Question, Tags, or Reports.
- ❖ In Manage Questions, use the Add button to create questions with options, answer, difficulty, and tags.
- ❖ In Manage Tags, add new topic tags or delete unused ones.
- ❖ View Reports for platform-wide student and attempt statistics.

Chapter 5 : RESULTS AND DISCUSSIONS

5.1 Home / Landing Page

The home page features an animated hero section with a grid background pattern, a pulsing 'Live' badge, and a large heading with gradient text. Statistics summary cards display 500+ Questions, 8 Domains, and 3 Difficulty Tiers. Feature cards explain the platform's capabilities. Navigation links provide Login and Get Started calls-to-action.

Figure 5.1: Home / Landing Page

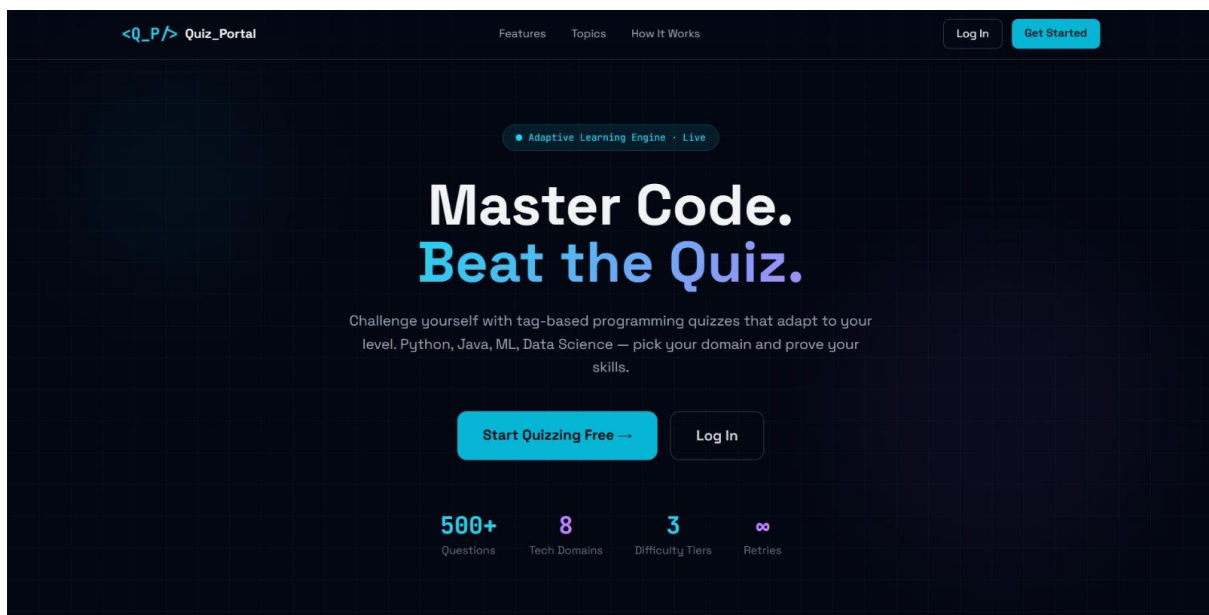
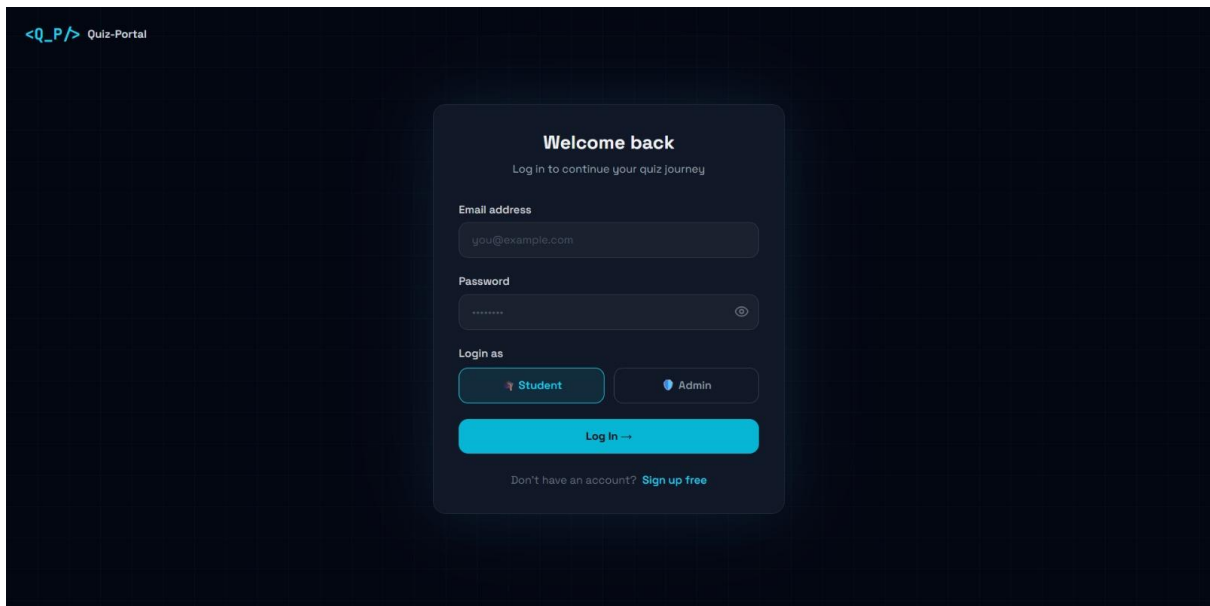


Figure 5.1: Online Quiz Portal Home Page with Hero Section and Feature Cards

5.2 Authentication Pages

The login page features a card-style form with dark background, email/password fields, and role-aware redirection (students to dashboard, admins to admin panel). The registration page includes real-time client-side validation and clear error messaging for duplicate emails or short passwords.

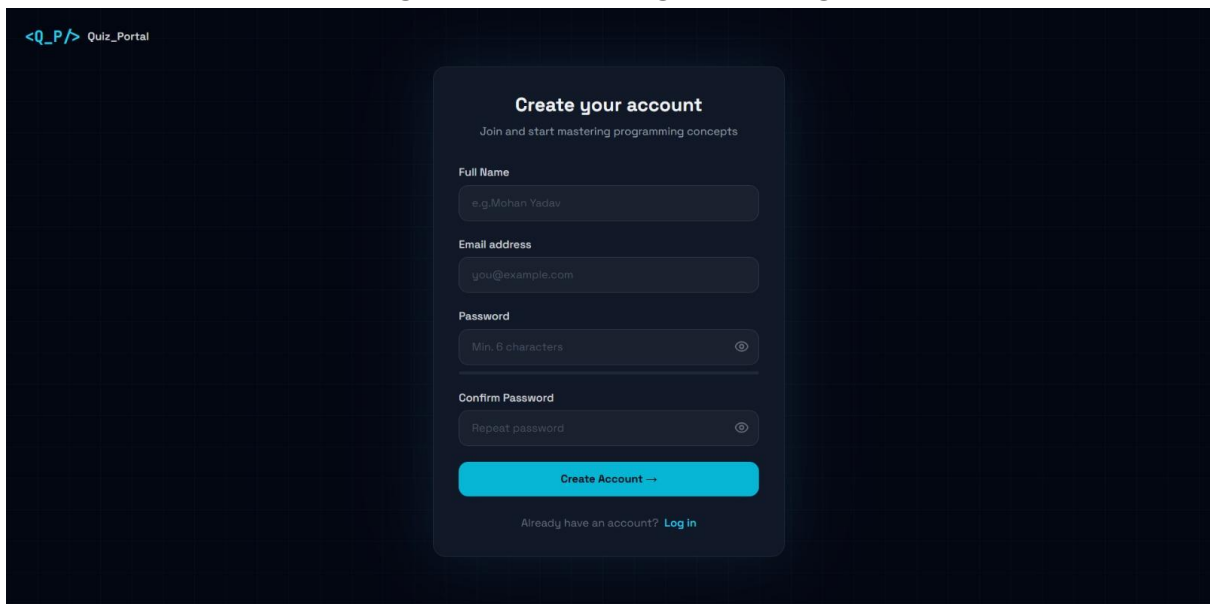
Figure 5.2: Student Login Page



The screenshot shows a dark-themed login page for a 'Quiz-Portal'. In the top-left corner, there is a logo consisting of a teal icon and the text '<Q_P/> Quiz-Portal'. The main content is a light gray card with rounded corners. At the top of the card, it says 'Welcome back' in bold, followed by the subtitle 'Log in to continue your quiz journey'. Below this are two input fields: 'Email address' with the placeholder 'you@example.com' and 'Password' with a placeholder 'password' and a toggle icon. Under the password field is a 'Login as' section with two buttons: 'Student' (with a student icon) and 'Admin' (with an admin icon). A large teal button labeled 'Log in →' is positioned below these. At the bottom of the card, it says 'Don't have an account? [Sign up free](#)'.

Figure 5.2: Login Page with Card-Style Form and Dark Background

Figure 5.3: Student Registration Page



The screenshot shows a dark-themed registration page for a 'Quiz-Portal'. In the top-left corner, there is a logo consisting of a teal icon and the text '<Q_P/> Quiz-Portal'. The main content is a light gray card with rounded corners. At the top of the card, it says 'Create your account' in bold, followed by the subtitle 'Join and start mastering programming concepts'. Below this are four input fields: 'Full Name' with the placeholder 'e.g. Mohan Yadav', 'Email address' with the placeholder 'you@example.com', 'Password' with the placeholder 'Min. 6 characters' and a toggle icon, and 'Confirm Password' with the placeholder 'Repeat password' and a toggle icon. A large teal button labeled 'Create Account →' is positioned below these. At the bottom of the card, it says 'Already have an account? [Log in](#)'.

Figure 5.3: Registration Page with Client-Side Validation

5.3 Student Dashboard

The student dashboard displays four key statistics: Total Attempts, Average Score, Best Score, and Topics Attempted. A purple resume banner appears at the top if an in-progress quiz is detected in the database. Recent quiz attempts are listed in card format with score progress bars. The sidebar provides navigation to Take a Quiz and My Attempts.

Figure 5.4: Student Dashboard with Resume Banner

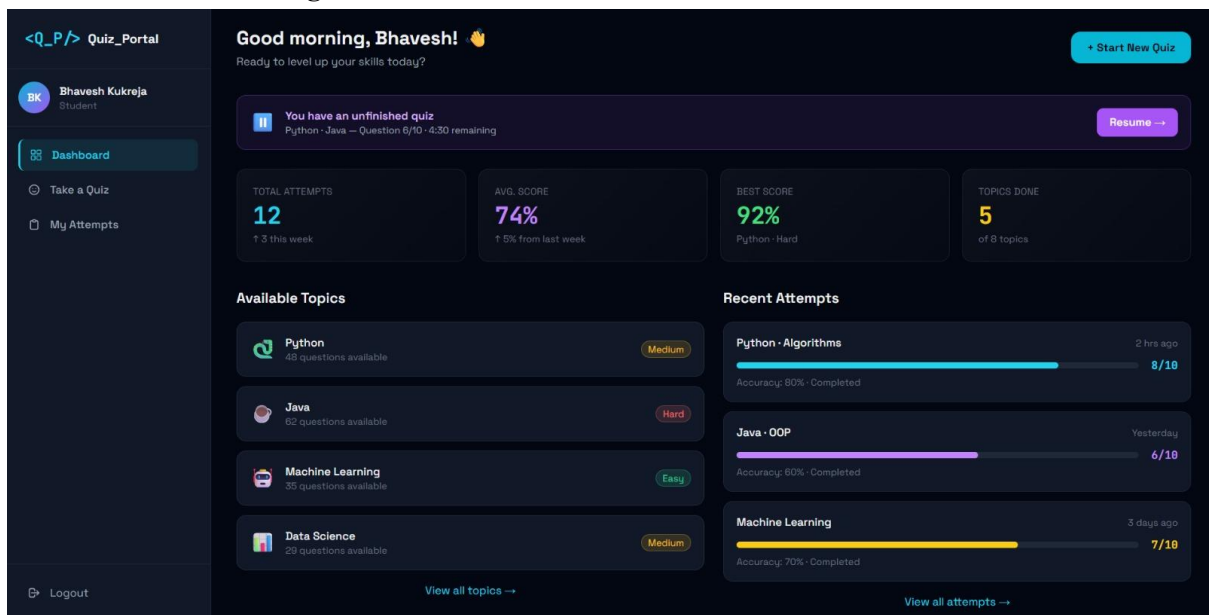


Figure 5.4: Student Dashboard with Stats Cards, Resume Banner, and Recent Attempts

5.4 Tag Selection Page

Students select from 8 topic tags presented as interactive cards with emoji icons and question counts. Cards toggle between selected and unselected states with a cyan border animation and checkmark icon. An information bar reminds students about adaptive difficulty and proctoring before quiz start.

Figure 5.5: Tag Selection Page

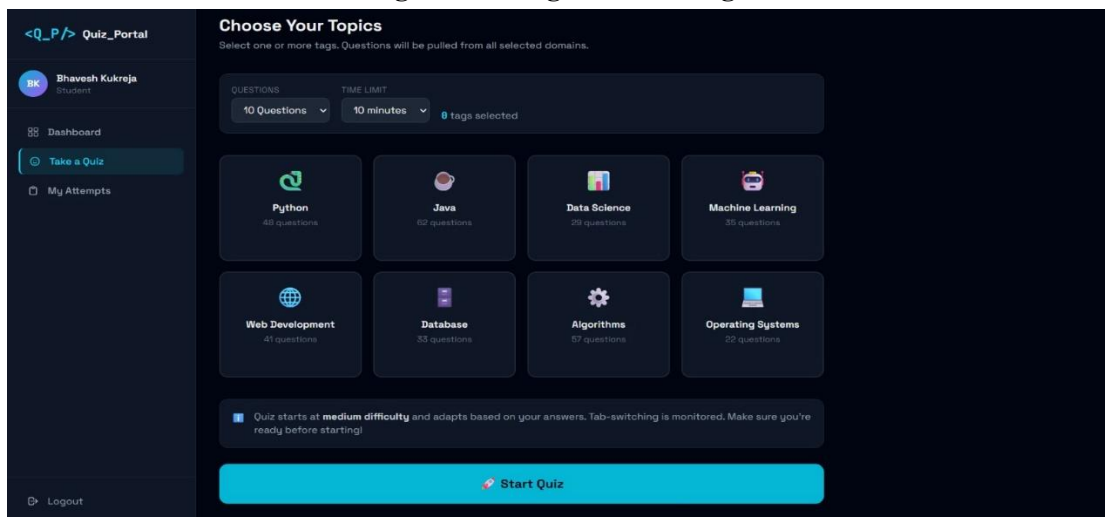


Figure 5.5: Tag Selection Page with 8 Topic Cards and Selection Animation

5.5 Quiz Page

The quiz page features a sticky header bar showing the countdown timer, three warning dots (turning red on violation), and a Submit Quiz button. A gradient progress bar below the header tracks completion. Questions are displayed in a dark card with four option buttons that highlight on selection. A difficulty badge (Easy/Medium/Hard) is shown for each question. An auto-saved indicator appears at the bottom left.

Figure 5.6: Quiz Page – Question with Timer and Proctoring Warnings

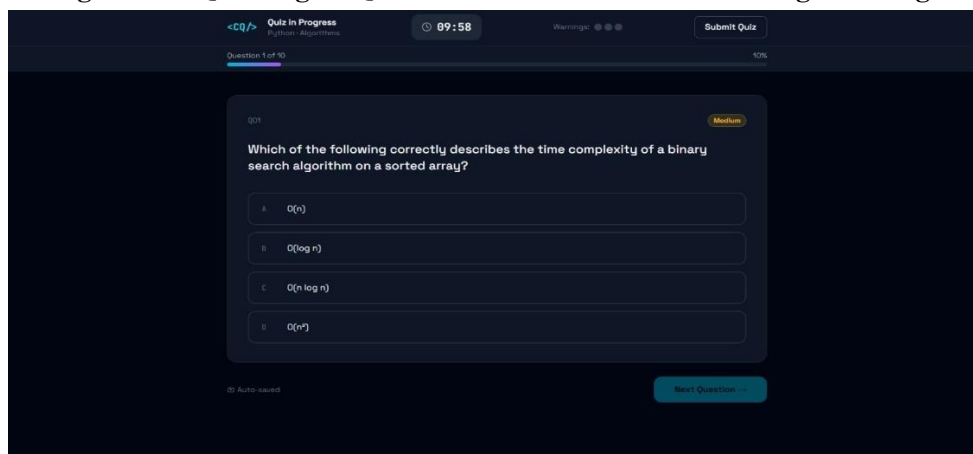


Figure 5.6: Quiz Page with Countdown Timer, Warning Dots, and Option Selection

5.6 Result Page

Results are displayed with an animated circular progress ring showing the score fraction. Below the ring, three cards display Correct answers, Incorrect answers, and Accuracy percentage. A complete question-by-question breakdown follows, with green left-border for correct answers and red left-border for incorrect ones. Wrong answers show the student's selection alongside the correct answer.

Figure 5.7: Result Page with Score Ring and Question Breakdown

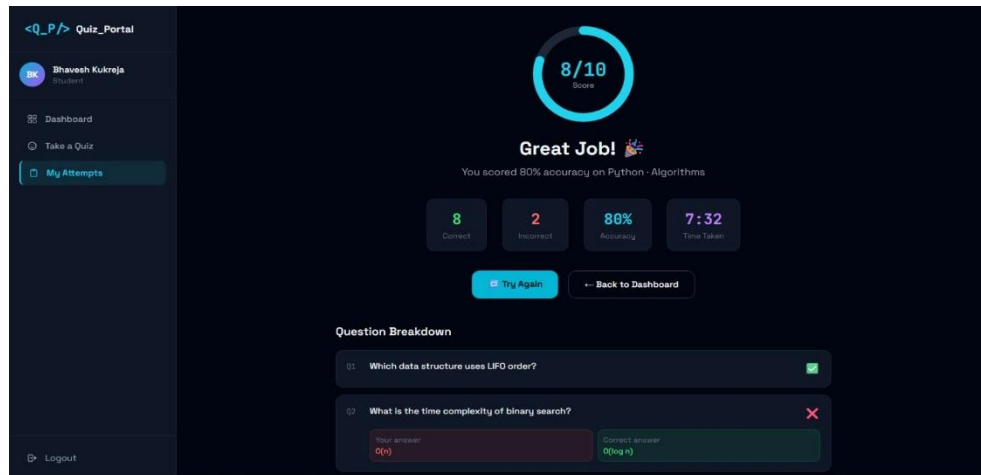


Figure 5.7.1: Result Page with Animated Score Ring, Accuracy Stats

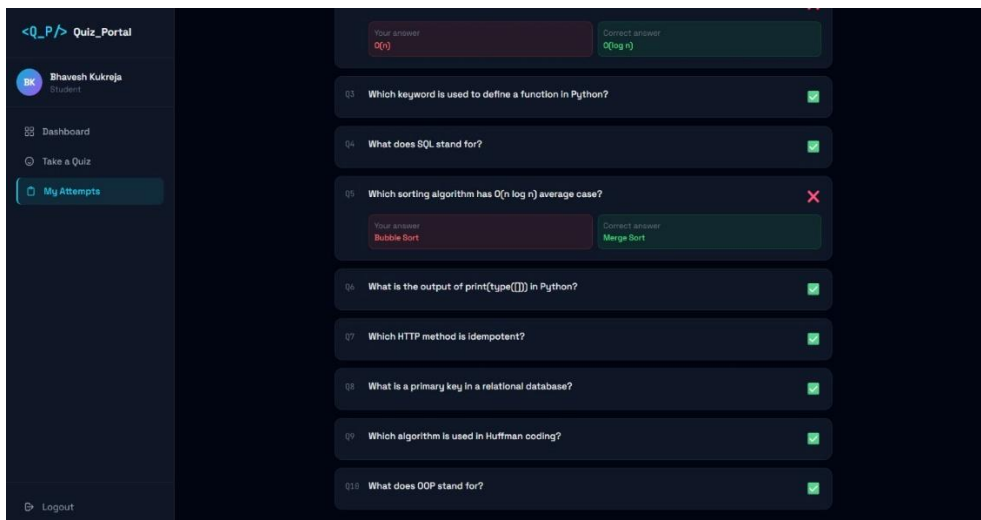


Figure 5.7.2: Result Page with Question Breakdowns

5.7 Admin Dashboard

The admin dashboard uses purple accent colors to visually distinguish it from the student interface. Four metric cards display Total Students, Total Questions, Total Attempts, and Total Tags. Horizontal bar charts show quiz attempts by topic. Quick action cards provide one-click navigation to Add Question, Manage Tags, All Questions, and Reports. A recent activity feed shows the latest student quiz events.

Figure 5.8: Admin Dashboard



Figure 5.8: Admin Dashboard with Purple-Accented Stats Cards and Topic Bar Charts

5.8 Admin Question Management

The questions management page displays all questions in a table with columns for question text (truncated), tags (color-coded badges), difficulty badge, and Edit/Delete actions. A modal form handles both Add and Edit operations with text fields for all four options, a correct answer dropdown, a difficulty dropdown, and tag checkboxes. Delete operations show a confirmation modal before execution.

Figure 5.9: Admin Manage Questions Page with Add/Edit Modal

The Admin Manage Questions page displays a table of 327 total questions. The table has columns for #, QUESTION, TAGS, DIFFICULTY, and ACTIONS. The ACTIONS column contains Edit and Delete links. A search bar and filters for All Tags and All Difficulties are at the top. An Add Question button is in the top right corner.

#	QUESTION	TAGS	DIFFICULTY	ACTIONS
1	What is the time complexity of binary search?	Python Algorithms	Medium	Edit Delete
2	Which keyword is used to create a class in Java?	Java	Easy	Edit Delete
3	What is overfitting in machine learning?	Machine Learning Data Science	Medium	Edit Delete
4	What does SQL GROUP BY clause do?	Database	Easy	Edit Delete
5	Explain the difference between GET and POST HTTP methods.	Web Development	Easy	Edit Delete
6	What is the difference between a stack and a queue?	Algorithms Python	Easy	Edit Delete
7	What is a lambda function in Python?	Python	Medium	Edit Delete
8	Describe deadlock in operating systems.	Operating Systems	Hard	Edit Delete

Figure 5.9: Admin Questions Page with Table and Add/Edit Modal Form

5.9 Module Performance Summary

Table 5.1: Module Performance Summary

Module	Metric Measured	Result
Tag-Based Question Fetch	10 questions via JDBC JOIN	<100ms for 96-question database
Auto-Save	AJAX POST to /quiz/save	<1 second; invisible to student
Adaptive Logic	Question.harder()/easier() + DB update	<5ms server-side computation
Quiz Start	Full session + progress creation	<300ms including DB writes
Proctoring Detection	Tab-switch event + toast display	<50ms (browser event listener)
Result Calculation	Score + accuracy from attempts	<200ms for 10-question quiz
Admin Question List	96 questions with tags via JOIN	<200ms
BCrypt Verification	Password verify on login	~300ms (cost factor 10, by design)

Chapter 6 SUMMARY AND CONCLUSIONS

6.1 Summary

The Online Quiz Portal was successfully designed, implemented, and tested as a full-stack Java web application for adaptive programming knowledge assessment. The system moved beyond a basic CRUD application to deliver a working adaptive quiz engine, real-time proctoring, auto-save and resume, a comprehensive admin panel, and a polished responsive UI — all within a clean MVC architecture.

Key achievements include: (i) A tag-based adaptive quiz engine supporting 8 programming domains with 96 seed questions; (ii) A three-tier adaptive difficulty system adjusting question level in real time after every answer; (iii) An auto-save mechanism persisting quiz state every 15 seconds via AJAX enabling complete resume functionality; (iv) Browser-based proctoring using the Page Visibility API and Fullscreen API with automatic quiz submission on 3 violations; (v) BCrypt password hashing with RBAC implemented via Java Servlet Filters; and (vi) A MySQL database with 6 normalized tables enforcing referential integrity through foreign key constraints.

6.2 Team Contributions

Member	Key Contributions
Bhavesh Kukreja	Backend Java architecture, Servlet design, DAO layer, MySQL schema, BCrypt/session security, Adaptive quiz engine, auto-save/resume, QuizDAO, UPSERT strategy, adaptive difficulty logic
Chandan Kawadker	All JSP views, Tailwind CSS frontend, JavaScript timer and proctoring, admin panel UI

6.3 Conclusions

The following conclusions are drawn from the development and evaluation of the Online Quiz Portal:

- ❖ The MVC architecture with Java Servlets and JSP effectively handles the full request-response cycle for a multi-user web application of this complexity.
- ❖ Adaptive difficulty using a three-tier system (Easy/Medium/Hard) significantly improves the assessment quality over fixed-difficulty quizzes by dynamically matching question difficulty to student ability.

- ❖ Browser-based proctoring using the Page Visibility and Fullscreen APIs is a lightweight, zero-installation alternative to heavy proctoring software, effective for academic environments.
- ❖ Auto-save with MySQL UPSERT (ON DUPLICATE KEY UPDATE) provides a reliable and computationally inexpensive mechanism for preserving quiz state every 15 seconds.
- ❖ BCrypt password hashing with a cost factor of 10 provides adequate security for user credentials while maintaining an acceptable authentication response time.
- ❖ Tailwind CSS enables rapid development of consistent, responsive, and visually appealing dark-themed interfaces without writing extensive custom CSS.
- ❖ All 15 defined test cases passed successfully, confirming functional correctness of all system modules.

Chapter 7 FUTURE SCOPE

The current Online Quiz Portal provides a robust foundation for future enhancements. The following extensions are planned for future development iterations:

- ❖ Webcam-Based Proctoring: Integration of TensorFlow.js face detection to monitor for face absence, multiple faces, and suspicious head movements during quizzes.
- ❖ Computerized Adaptive Testing (IRT): Replacement of the current heuristic harder/easier logic with a proper Item Response Theory model for more accurate student ability estimation.
- ❖ Leaderboard and Gamification: A public student leaderboard, achievement badges, and streak tracking to increase engagement and motivation.
- ❖ AI Question Generation: Integration with a large language model (LLM) API to auto-generate MCQ questions for new topics, reducing manual admin effort.
- ❖ REST API and Mobile App: Migration of the quiz engine to a REST API backend to support native Android and iOS mobile applications via the same MySQL backend.
- ❖ Detailed Analytics Dashboard: Per-tag performance graphs, score improvement trends over time, and peer comparison analytics for individual students.
- ❖ Email Notifications: Automated email on registration, quiz completion, and weekly performance summary using the JavaMail API.
- ❖ Code Snippet Questions: Support for syntax-highlighted code snippet questions using a JavaScript code highlighting library, making assessments more realistic for programming topics.
- ❖ PDF Result Export: Allow students to download their quiz results as formatted PDF reports using Apache PDFBox.
- ❖ Multi-Language Support (i18n): Internationalization support for Hindi and regional language interfaces to make the platform accessible to a wider student population.

Appendix A – Glossary

Term	Definition
Online Quiz Portal	The full-stack Java web application described in this report
MVC	Model-View-Controller – architectural pattern separating data, UI, and control logic
Adaptive Quiz	Quiz that adjusts question difficulty based on student performance in real time
BCrypt	Blowfish-based password hashing algorithm with configurable cost factor
AuthFilter	Java Servlet Filter that blocks unauthenticated access to protected URLs
UPSERT	MySQL ON DUPLICATE KEY UPDATE – insert or update in a single atomic operation
UUID	Universally Unique Identifier – 128-bit random ID used as quiz session_id
AJAX	Asynchronous JavaScript and XML – used for 15-second auto-save without page reload
Page Visibility API	Browser API detecting when a tab becomes hidden (used for tab-switch proctoring)
Fullscreen API	Browser API for fullscreen requests and exit detection (used for proctoring)
PreparedStatement	JDBC parameterized SQL statement preventing SQL injection attacks
WAR	Web Application Archive – deployment package for Java web apps on Tomcat

Bibliography

- [1] D. J. Weiss, "Adaptive Testing by Computer," *Journal of Consulting and Clinical Psychology*, vol. 53, no. 6, pp. 774–789, 1985.
- [2] H. M. Alessio, N. Malay, K. Maurer, A. J. Bailer, and B. Rubin, "Examining the Effect of Proctoring on Online Test Scores," *Online Learning*, vol. 21, no. 1, pp. 146–161, 2017.
- [3] B. Eckel, *Thinking in Java*, 4th ed. Prentice Hall, 2006.
- [4] M. Hall and L. Brown, *Core Servlets and JavaServer Pages*. Prentice Hall PTR, 2004.
- [5] P. DuBois, *MySQL*, 5th ed. Addison-Wesley Professional, 2013.
- [6] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [7] Apache Tomcat Documentation. [Online]. Available: <https://tomcat.apache.org/tomcat-9.0-doc/>. [Accessed: April 2026].
- [8] Oracle Java SE 21 Documentation. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/>. [Accessed: April 2026].
- [9] MySQL 8.0 Reference Manual. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>. [Accessed: April 2026].
- [10] Tailwind CSS Documentation. [Online]. Available: <https://tailwindcss.com/docs>. [Accessed: April 2026].
- [11] jBCrypt Library. [Online]. Available: <https://www.mindrot.org/projects/jBCrypt/>. [Accessed: April 2026].
- [12] Gson User Guide. [Online]. Available: <https://github.com/google/gson/blob/main/UserGuide.md>. [Accessed: April 2026].
- [13] MDN Web Docs – Page Visibility API. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Page_Visibility_API. [Accessed: April 2026].
- [14] MDN Web Docs – Fullscreen API. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Fullscreen_API. [Accessed: April 2026].
- [15] Apache Maven Documentation. [Online]. Available: <https://maven.apache.org/guides/>. [Accessed: April 2026].